

COMPUTER VISION LAB MID - PRINTABLE CHEATSHEET

Labs 01-06 | Python + NumPy + Matplotlib + OpenCV | Open-book lab exam

Built to be searched visually under exam pressure: copy a template, change image path / parameters, run, inspect, then tune.

Need	Use
Read / show image	cv2.imread, cvtColor, plt.imshow
ROI / overlay / masks	NumPy slicing, addWeighted, bitwise_*
Threshold / document	threshold, Otsu, adaptiveThreshold
Geometric transform	resize, warpAffine, getRotationMatrix2D, warpPerspective
Filtering / features	filter2D, GaussianBlur, Sobel, Canny, HOG/LBP/hist
Segmentation	HSV mask, region growing, watershed, k-means
Lines / circles	HoughLinesP, HoughCircles
Matching / panorama	SIFT, BFMatcher/FLANN, findHomography, warpPerspective
Video	VideoCapture, VideoWriter, frame loop

Universal imports + display

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

img = cv2.imread("image.jpg")
gray = cv2.imread("image.jpg", cv2.IMREAD_GRAYSCALE) # BGR
if img is None:
    raise FileNotFoundError("Image not found")

rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(rgb); plt.axis("off"); plt.show()

# shape: color=(H,W,3), gray=(H,W)
h, w = img.shape[:2]
print(img.shape, img.dtype, img.min(), img.max(), img.mean())
```

Exam note: OpenCV stores color as BGR. Matplotlib expects RGB. OpenCV sizes are usually (width, height), while NumPy image shape is (height, width, channels).

LAB 01 - Python + OpenCV Fundamentals

Python essentials: list/dict/functions/classes

```
# list / loop
vals = [10, 20, 30]
for x in vals:
    print(x)

# dictionary
students = {
    73: {"Name": "Rayyan", "Major": "AI", "Grades": [88, 86, 92, 91]},
    74: {"Name": "Malik", "Major": "AI", "Grades": [90, 87, 94, 89]}
}
for roll, s in students.items():
    avg = sum(s["Grades"]) / len(s["Grades"])
    print(roll, s["Name"], avg)

def search_major(students, major):
    for s in students.items():
        if s["Major"] == major:
            print(s["Name"])

class GroceryManager:
    def __init__(self):
        self.items = []

    def add_item(self, item, quantity, price):
        self.items.append({'item': item, 'quantity': quantity, 'price': price})

    def remove_item(self, item):
        for x in self.items:
            if x["item"] == item:
                self.items.remove(x); return

    def total(self):
        return sum(x["quantity"]*x["price"] for x in self.items)

# Create canvas + draw
canvas = np.zeros((800, 800, 3), dtype=np.uint8)
center = (400, 400)

cv2.line(canvas, (50, 50), (750, 50), (0, 255, 0), 3)
cv2.rectangle(canvas, (150, 150), (650, 650), (0, 0, 255), 4)
cv2.rectangle(canvas, (50, 650), (300, 780), (255, 0, 0), -1) # filled
cv2.circle(canvas, center, 250, (255, 255, 255), 2)
cv2.circle(canvas, center, 50, (0, 255, 255), -1)

cv2.putText(canvas, "OpenCV", (50, 100),
            cv2.FONT_HERSHEY_SIMPLEX, 1.2, (255, 255, 255), 2)

plt.imshow(cv2.cvtColor(canvas, cv2.COLOR_BGR2RGB))
plt.axis("off"); plt.show()

# ROI + blur
blurred = cv2.GaussianBlur(img, (25, 25), 0) # kernel dimensions must be odd
h, w = img.shape[:2]
cx, cy = w//2, h//2
half = 150
x1, x2 = cx-half, cx+half
y1, y2 = cy-half, cy+half

roi = img[y1:y2, x1:x2]
roi_blur = blurred[y1:y2, x1:x2] # NumPy indexing = [y, x]

# Transparent overlay + text
overlay = img.copy()
h, w = img.shape[:2]
start_y = int(h*0.80)
cv2.rectangle(overlay, (0, start_y), (w, h), (255, 0, 0), -1)
result = cv2.addWeighted(img, 0.7, overlay, 0.3, 0)
cv2.putText(result, "Hello", (50, h-50),
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 3)
```

Global vs adaptive threshold + rotate

```
gray = cv2.imread("document.png", cv2.IMREAD_GRAYSCALE)

_, global_th = cv2.threshold(gray, 127, 255, cv2.THRESH_BINARY)

adaptive = cv2.adaptiveThreshold(
    gray, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

h,w = adaptive.shape
M = cv2.getRotationMatrix2D((w//2,h//2), 45, 0.8)
rotated = cv2.warpAffine(adaptive, M, (w,h))

Circular mask + combine two images

a = cv2.imread("a.jpg")
b = cv2.imread("b.jpg")
h,w = a.shape[:2]
b = cv2.resize(b, (w,h))

mask = np.zeros((h,w), dtype=np.uint8)
cv2.circle(mask, (w//2,h//2), min(w,h)//4, 255, -1)
```

```
fg = cv2.bitwise_and(a, a, mask=mask)
inv = cv2.bitwise_not(mask)
bg = cv2.bitwise_and(b, b, mask=inv)
combined = cv2.bitwise_or(fg, bg)
```

Pixel channel statistics

```
import pandas as pd
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
r = rgb[:, :, 0].flatten()
g = rgb[:, :, 1].flatten()
b = rgb[:, :, 2].flatten()
df = pd.DataFrame({"Red":r, "Green":g, "Blue":b})
print(df.describe())
```

LAB 02 - Image Enhancement, Fusion, Video

Histogram + equalization

```
gray = cv2.imread("xray.png", cv2.IMREAD_GRAYSCALE)

plt.hist(gray.ravel(), bins=256, range=[0,256])
plt.xlabel("Intensity"); plt.ylabel("Pixels"); plt.show()

eq = cv2.equalizeHist(gray)
plt.hist(eq.ravel(), 256, (0,256)); plt.show()

Heatmap / pseudo-color

heat = cv2.applyColorMap(eq, cv2.COLORMAP_JET) # also HOT, VIRIDIS, TURBO
plt.imshow(cv2.cvtColor(heat, cv2.COLOR_BGR2RGB))
plt.axis("off"); plt.show()
```

Gray-world color balance

```
def gray_world_balance(image):
    x = image.astype(np.float32)
    b,g,r = cv2.split(x)
    mb,mg,mr = np.mean(b),np.mean(g),np.mean(r)
    mean_gray = (mb+mg+mr)/3.0
    eps = 1e-6
    b *= mean_gray/(mb+eps)
    g *= mean_gray/(mg+eps)
    r *= mean_gray/(mr+eps)
    return np.clip(cv2.merge([b,g,r]),0,255).astype(np.uint8)

balanced = gray_world_balance(heat)
```

balanced = gray_world_balance(heat)

Threshold high-intensity regions

```
T = 200
_, mask = cv2.threshold(eq, T, 255, cv2.THRESH_BINARY)
filtered = cv2.bitwise_and(eq, eq, mask=mask)
```

Log and gamma transforms

```
def log_transform(image):
    x = image.astype(np.float32)
    mx = np.max(x)
    if mx == 0: return np.zeros_like(image)
    c = 255.0 / np.log1p(mx)
    return np.clip(c*np.log1p(x),0,255).astype(np.uint8)

def gamma_transform(image, gamma=0.6):
    x = image.astype(np.float32)/255.0
    y = np.power(x, gamma)*255.0
    return np.clip(y,0,255).astype(np.uint8)

log_img = log_transform(gray)
gamma_img = gamma_transform(gray, 0.6)
```

Exam note: Gamma < 1 brightens dark regions; gamma > 1 darkens/suppresses bright regions.

CT + MRI weighted fusion

```
ct = cv2.imread("ct.png", cv2.IMREAD_GRAYSCALE)
mri = cv2.imread("mri.png", cv2.IMREAD_GRAYSCALE)
if ct.shape != mri.shape:
    mri = cv2.resize(mri, (ct.shape[1], ct.shape[0]))

ct_eq, mri_eq = cv2.equalizeHist(ct), cv2.equalizeHist(mri)
ct_c = cv2.applyColorMap(ct_eq, cv2.COLORMAP_JET)
mri_c = cv2.applyColorMap(mri_eq, cv2.COLORMAP_HOT)

fused = cv2.addWeighted(ct_c, 0.7, mri_c, 0.3, 0)
cv2.imwrite("fusion.png", fused)

Video capture / properties / correct writer

cap = cv2.VideoCapture("input.mp4")
if not cap.isOpened():
    raise FileNotFoundError("Could not open video")
```

```

fps = cap.get(cv2.CAP_PROP_FPS) or 30
w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print(fps, w, h, n, "duration=", n/fps)

fourcc = cv2.VideoWriter_fourcc(*"mp4v")
writer = cv2.VideoWriter("out.mp4", fourcc, fps, (w*2, h))

while True:
    ok, frame = cap.read()
    if not ok: break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    eq = cv2.equalizeHist(gray)
    enhanced = cv2.applyColorMap(eq, cv2.COLORMAP_JET)

    left = frame.copy()
    right = enhanced.copy()
    cv2.putText(left, "RAW", (20,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)
    cv2.putText(right, "ENHANCED", (20,40), cv2.FONT_HERSHEY_SIMPLEX, 1, (255,255,255), 2)

    combined = np.hstack((left, right)) # exactly matches (w*2, h)
    writer.write(combined)

cap.release()
writer.release()
cv2.destroyAllWindows()

```

Exam note: Writer frame size **MUST** exactly match every frame you write. Do not create a (w,h) writer and then write np.hstack(...), which is (2w,h).

LAB 03 - Geometric Transformations

Coordinate / matrix facts

Pixel: image[y,x]. Origin is top-left; +x goes right; +y goes down. **Affine:** parallel lines stay parallel. uses 2x3 matrix and warpAffine. **Perspective/projective:** uses 3x3 homography and warpPerspective.

Resize / scale

```
h,w = img.shape[:2]

# explicit size
small_img = cv2.resize(img, (w//2, h//2), interpolation=cv2.INTER_AREA)

# scale factors
bigger = cv2.resize(img, None, fx=1.5, fy=1.5,
                    interpolation=cv2.INTER_LINEAR)
# INTER_AREA: shrink; INTER_LINEAR: common enlarge; INTER_CUBIC: higher quality
```

Translation

```
h,w = img.shape[:2]
tx, ty = 100, 50
M = np.float32([[1,0,tx],
               [0,1,ty]])
translated = cv2.warpAffine(img, M, (w,h))
```

Rotation

```
h,w = img.shape[:2]
center = (w//2,h//2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rot = cv2.warpAffine(img, M, (w,h))
```

Shear

```
h,w = img.shape[:2]
M = np.float32([[1,0.3,0],
               [0.2,1,0]])
sheared = cv2.warpAffine(img, M, (w,h))
```

Flip

```
horizontal = cv2.flip(img, 1)
vertical   = cv2.flip(img, 0)
both       = cv2.flip(img, -1)
```

Affine transform from 3 point pairs

```
h,w = img.shape[:2]
src = np.float32([[50,50],[200,50],[50,200]])
dst = np.float32([[10,100],[220,50],[100,250]])
```

```
M = cv2.getAffineTransform(src, dst)
affine = cv2.warpAffine(img, M, (w,h))
```

Perspective transform / document rectification

```
src = np.float32([[x1,y1],[x2,y2],[x3,y3],[x4,y4]]) # TL,TR,BR,BL
dst = np.float32([[0,0],[W-1,0],[W-1,H-1],[0,H-1]])
```

```
Hmat = cv2.getPerspectiveTransform(src, dst)
rectified = cv2.warpPerspective(img, Hmat, (W,H))
```

Transform composition with homogeneous 3x3 matrices

```
T = np.array([[1,0,tx],
             [0,1,ty],
             [0,0,1]], dtype=np.float32)
```

```
theta = np.deg2rad(30)
R = np.array([[np.cos(theta),-np.sin(theta),0],
             [np.sin(theta), np.cos(theta),0],
             [0,0,1]], dtype=np.float32)
```

```
C = T @ R # order matters
out = cv2.warpPerspective(img,C, (w,h))
```

Exam note: Matrix multiplication order matters. A @ B means B acts first on column-vector coordinates.

LAB 04 - Feature Extraction, Filtering, Convolution, Canny

Manual 2D convolution intuition

Kernel slides over image; output pixel is weighted sum of neighborhood. Padding controls borders. Stride controls movement. OpenCV filter2D does not have a stride argument; to emulate stride, filter normally then subsample.

filter2D + common kernels

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# averaging
k = np.ones((3,3), np.float32)/9
avg = cv2.filter2D(gray, -1, k)

# sharpening
sharp_k = np.array([[0,-1,0],
                    [-1,5,-1],
                    [0,-1,0]], np.float32)
sharp = cv2.filter2D(gray, -1, sharp_k)

# edge kernel
edge_k = np.array([[[-1,-1,-1],
                    [-1, 8,-1],
                    [-1,-1,-1]], np.float32)
edge = cv2.filter2D(gray, cv2.CV_32F, edge_k)

# emulate stride=2 after valid/same filtering:
stride2 = avg[::2, ::2]
```

Blur / denoise

```
mean_blur = cv2.blur(img, (5,5))
gauss = cv2.GaussianBlur(img, (5,5), 0)
median = cv2.medianBlur(img, 5)
bilateral = cv2.bilateralFilter(img, 9, 75, 75) # strong for salt-pepper
# smooth while preserving edges
```

Sobel gradients + magnitude / direction

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)

mag = cv2.magnitude(gx.astype(np.float32), gy.astype(np.float32))
angle = cv2.phase(gx.astype(np.float32), gy.astype(np.float32),
                  angleInDegrees=True)
```

```
# display absolute gradient
sx = cv2.convertScaleAbs(gx)
sy = cv2.convertScaleAbs(gy)
combined = cv2.addWeighted(sx, 0.5, sy, 0.5, 0)
```

Canny edge detector

```
blur = cv2.GaussianBlur(gray, (5,5), 1.4)
edges = cv2.Canny(blur, 50, 150)

# tune low/high thresholds
e1 = cv2.Canny(gray, 30, 100)
e2 = cv2.Canny(gray, 75, 150)
e3 = cv2.Canny(gray, 100, 200)
```

Canny pipeline: Gaussian smoothing -> gradients -> magnitude/direction -> non-maximum suppression -> double threshold -> hysteresis.

Color histogram feature

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
features = []
for i in range(3):
    hist = cv2.calcHist([rgb], [i], None, [256], [0, 256])
    hist = cv2.normalize(hist, hist).flatten()
    features.extend(hist)
features = np.array(features)
```

Texture: LBP (basic implementation)

```
def lbp_basic(gray):
```

```

h,w = gray.shape
out = np.zeros_like(gray)
offsets = [(-1,-1),(-1,0),(-1,1),(0,1),
           (1,1),(1,0),(1,-1),(0,-1)]
for y in range(1,h-1):
    for x in range(1,w-1):
        c = gray[y,x]
        code = 0
        for bit,(dy,dx) in enumerate(offsets):
            code |= (gray[y+dy,x+dx] >= c) << bit
        out[y,x] = code
    return out

lbp = lbp_basic(gray)
lbp_hist,_ = np.histogram(lbp.ravel(), bins=256, range=(0,256), density=True)

HOG feature descriptor

# Common OpenCV HOG settings for 64x128 image
roi = cv2.resize(gray, (64,128))
hog = cv2.HOGDescriptor(
    _winSize=(64,128),
    _blockSize=(16,16),
    _blockStride=(8,8),
    _cellSize=(8,8),
    _nbins=9
)
hog_vec = hog.compute(roi).flatten()
print(hog_vec.shape)

Texture energy / contrast from gradients

gx = cv2.Sobel(gray, cv2.CV_32F,1,0,ksize=3)
gy = cv2.Sobel(gray, cv2.CV_32F,0,1,ksize=3)
mag = cv2.magnitude(gx,gy)
energy = np.mean(mag**2)
contrast = np.std(gray)
print("energy", energy, "contrast", contrast)

```

LAB 05 - Segmentation: Thresholds, Color, Canny, Region Growing, Watershed, K-Means

Task 1 - Global threshold sweep + adaptive

```
gray = cv2.imread("document.png", cv2.IMREAD_GRAYSCALE)

thresholds = [80, 120, 160]
masks = []
for T in thresholds:
    →, m = cv2.threshold(gray, T, 255, cv2.THRESH_BINARY)
    masks.append(m)

adaptive = cv2.adaptiveThreshold(
    gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY, 31, 10
)
```

Task 2 - Adaptive threshold parameter study

```
# blockSize MUST be odd and > 1
for block in [11, 31, 51]:
    for C in [2, 7, 12]:
        mean_m = cv2.adaptiveThreshold(
            gray, 255, cv2.ADAPTIVE_THRESH_MEAN_C,
            cv2.THRESH_BINARY, block, C)
        gauss_m = cv2.adaptiveThreshold(
            gray, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
            cv2.THRESH_BINARY, block, C)
```

Small block: very local/noisy. Large block: smoother, may miss local illumination. Larger C subtracts more from local mean -> changes foreground amount.

Task 3 - Otsu automatic threshold

```
otsu_T, otsu = cv2.threshold(
    gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
print("Otsu threshold:", otsu_T)

plt.hist(gray.ravel(), 256, [0, 256])
plt.axvline(otsu_T, linestyle="--")
plt.show()
```

Task 4 - HSV color segmentation

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# choose/tune these from HSV values
lower = np.array([35, 60, 50])
upper = np.array([85, 255, 255])

mask = cv2.inRange(hsv, lower, upper)
result = cv2.bitwise_and(img, img, mask=mask)

# clean mask
kernel = np.ones((5, 5), np.uint8)
mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, kernel)
mask = cv2.morphologyEx(mask, cv2.MORPH_CLOSE, kernel)
```

Exam note: HSV in OpenCV: H=0..179, S=0..255, V=0..255. For red, use two hue ranges near 0 and 179 and OR the masks.

Task 5 - Canny threshold study

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (5, 5), 0)

edges1 = cv2.Canny(blur, 30, 100)
edges2 = cv2.Canny(blur, 75, 150)
edges3 = cv2.Canny(blur, 120, 240)
```

Task 6 - Region growing

```
from collections import deque

def region_grow(gray, seed, threshold=12):
    h, w = gray.shape
    sx, sy = seed
    mask = np.zeros((h, w), np.uint8)
    # seed as (x, y)
```

```

q = deque([(sx,sy)])
seed_val = int(gray[sy,sx])
mask[sy,sx] = 255

while q:
    x,y = q.popleft()
    for dx,dy in [(1,0),(-1,0),(0,1),(0,-1)]:
        nx,ny = x+dx,y+dy
        if 0 <= nx < w and 0 <= ny < h and mask[nx,ny] == 0:
            if abs(int(gray[nx,ny])-seed_val) <= threshold:
                mask[nx,ny] = 255
                q.append((nx,ny))
    return mask

mask = region_grow(gray, seed=(w//2,h//2), threshold=15)
seg = cv2.bitwise_and(img,mask)

```

LAB 05 continued - Watershed + K-Means + Method Comparison

Task 7 - Watershed for touching coins

```
img = cv2.imread("coins.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, th = cv2.threshold(gray, 0, 255,
                      cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

kernel = np.ones((3, 3), np.uint8)
opening = cv2.morphologyEx(th, cv2.MORPH_OPEN, kernel, iterations=2)
sure_bg = cv2.dilate(opening, kernel, iterations=3)
```

```
dist = cv2.distanceTransform(opening, cv2.DIST_L2, 5)
_, sure_fg = cv2.threshold(dist, 0.5*dist.max(), 255, 0)
sure_fg = np.uint8(sure_fg)
```

```
unknown = cv2.subtract(sure_bg, sure_fg)
```

```
num_labels, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0
```

```
markers = cv2.watershed(img, markers)
result = img.copy()
result[markers == -1] = [0, 0, 255] # boundaries red
```

```
labels = np.unique(markers)
count = np.sum(labels > 1)
print("objects:", count)
```

Task 8 - Watershed distance threshold experiment

```
for factor in [0.30, 0.40, 0.50, 0.60]:
    _, fg = cv2.threshold(dist, factor*dist.max(), 255, 0)
    fg = np.uint8(fg)
    unknown = cv2.subtract(sure_bg, fg)
```

```
n, markers = cv2.connectedComponents(fg)
markers = markers + 1
markers[unknown == 255] = 0
```

```
ws = cv2.watershed(img.copy(), markers)
regions = np.sum(np.unique(ws) > 1)
print(factor, "FG markers:", n-1, "regions:", regions)
```

Task 9 - K-means image segmentation

```
def kmeans_segment(image, rgb, K):
    pixels = image_rgb.reshape((-1, 3)).astype(np.float32)

    criteria = (cv2.TERM_CRITERIA_EPS +
                 cv2.TERM_CRITERIA_MAX_ITER, 100, 0.2)

    compactness, labels, centers = cv2.kmeans(
        pixels, K, None, criteria, 10, cv2.KMEANS_RANDOM_CENTERS)

    centers = np.uint8(centers)
    segmented = centers[labels.flatten()].reshape(image_rgb.shape)
    return segmented, centers, labels, compactness
```

```
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
for K in [2, 4, 6]:
    seg, centers, labels, compactness = kmeans_segment(rgb, K)
    print(K, centers, compactness)
```

Task 10 - Unknown image: compare 3 segmentation methods

```
# Example set: global vs Otsu vs adaptive
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

_, m1 = cv2.threshold(gray, 150, 255, cv2.THRESH_BINARY_INV)
T, m2 = cv2.threshold(gray, 0, 255,
```

```

m3 = cv2.adaptiveThreshold(gray, 255,
                           cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                           cv2.THRESH_BINARY_INV, 31, 10)

def fg_percent(mask):
    return np.count_nonzero(mask)/mask.size*100

print("Global", fg_percent(m1))
print("Otsu", fg_percent(m2))
print("Adaptive", fg_percent(m3))

```

Pick method by scene: uneven illumination -> adaptive; bimodal grayscale -> Otsu; known stable intensity -> global; known color -> HSV; touching objects -> watershed; color-region simplification -> k-means; connected homogeneous seed region growing.

LAB 06 - Wavelets, Boundary Detection, Hough Transform, SIFT

Sobel / LoG / Canny boundaries

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Sobel
gx = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
mag = np.sqrt(gx**2 + gy**2)
mag8 = cv2.normalize(mag, None, 0, 255, cv2.NORM_MINMAX).astype(np.uint8)
```

```
# Laplacian of Gaussian
blur = cv2.GaussianBlur(gray, (5, 5), 0)
log_edge = cv2.Laplacian(blur, cv2.CV_64F)
log_edge = cv2.convertScaleAbs(log_edge)
```

```
# Canny
canny = cv2.Canny(blur, 50, 150)
```

Hough line transform - lane / screen detection

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
edges = cv2.Canny(gray, 50, 150)

lines = cv2.HoughLinesP(
    edges,
    rho=1,
    theta=np.pi/180,
    threshold=80,
    minLineLength=50,
    maxLineGap=20
)

out = img.copy()
if lines is not None:
    for line in lines[:10]:
        x1, y1, x2, y2 = line
        cv2.line(out, (x1, y1), (x2, y2), (0, 0, 255), 3)
```

Hough circles - coin detection/counting

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
gray = cv2.medianBlur(gray, 5)

circles = cv2.HoughCircles(
    gray, cv2.HOUGH_GRADIENT,
    dp=1.2, mindist=25,
    param1=100, param2=30,
    minRadius=10, maxRadius=100
)

out = img.copy()
count = 0
if circles is not None:
    circles = np.uint16(np.around(circles[0]))
    count = len(circles)
    for x, y, r in circles:
        cv2.circle(out, (x, y), r, (0, 255, 0), 2)
        cv2.circle(out, (x, y), 2, (0, 0, 255), 3)
    print("coins:", count)
```

Wavelet denoising / anomaly detection

```
import pywt

# 1D sensor signal
coeffs = pywt.wavedec(signal, "db4", level=3)
cA, *details = coeffs

# soft-threshold detail coefficients
sigma = np.median(np.abs(details[-1])) / 0.6745
thr = sigma*np.sqrt(2*np.log(len(signal)))
new_coeffs = [cA] + [pywt.threshold(d, thr, mode="soft") for d in details]
```

```

denoised = pywt.waerverc(new_coeffs, "db4")[:len(signal)]

residual = signal - denoised
anomaly_mask = np.abs(residual) > 3*np.std(residual)

plt.plot(signal, label="sensor")
plt.plot(denoised, label="denoised")
plt.scatter(np.where(anomaly_mask)[0],
            signal[anomaly_mask], label="anomaly")
plt.legend(); plt.show()

2D wavelet decomposition

coeffs2 = pywt.dwt2(gray.astype(np.float32), "haar")
cA, (cH, cV, cD) = coeffs2
recon = pywt.idwt2((cA, (cH, cV, cD)), "haar")

SIFT keypoints + descriptors

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
sift = cv2.SIFT_create()

kp, des = sift.detectAndCompute(gray, None)
vis = cv2.drawKeypoints(
    img, kp, None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)
print("keypoints:", len(kp), "descriptor shape:", None if des is None else des.shape)

```

LAB 06 continued - SIFT Matching, Object Recognition, Panorama, Security

SIFT matching with Lowe ratio test

```
sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

bf = cv2.BFMatcher(cv2.NORM_L2)
pairs = bf.knnMatch(des1, des2, k=2)

good = []
for m,n in pairs:
    if m.distance < 0.75*n.distance:
        good.append(m)
```

```
matched = cv2.drawMatches(
    img1, kp1, img2, kp2, good, None,
    flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
```

Object localization using homography

```
if len(good) >= 4:
    src_pts = np.float32([kp1[m.queryIdx].pt for m in good]).reshape((-1,1,2))
    dst_pts = np.float32([kp2[m.trainIdx].pt for m in good]).reshape((-1,1,2))

    Hmat, inlier_mask = cv2.findHomography(
        src_pts, dst_pts, cv2.RANSAC, 5.0)

    h,w = gray1.shape
    box = np.float32([[0,0],[w,0],[w,h],[0,h]]).reshape((-1,1,2))
    projected = cv2.perspectiveTransform(box, Hmat)
    scene_out = cv2.polylines(
        img2.copy(), [np.int32(projected)], True, (0,255,0), 3)
```

Video asset/object recognition

```
reference = cv2.imread("asset.jpg", cv2.IMREAD_GRAYSCALE)
sift = cv2.SIFT_create()
kpR, desR = sift.detectAndCompute(reference, None)
bf = cv2.BFMatcher(cv2.NORM_L2)

cap = cv2.VideoCapture("lab.mp4")
while True:
    ok, frame = cap.read()
    if not ok: break
    g = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    kpF, desF = sift.detectAndCompute(g, None)
    if desF is None: continue

    pairs = bf.knnMatch(desR, desF, k=2)
    good = [m for m,n in pairs if m.distance < 0.75*n.distance]

    if len(good) >= 8:
        cv2.putText(frame, "ASSET FOUND", (20,40),
            cv2.FONT_HERSHEY_SIMPLEX, 1, (0,255,0), 2)

    cap.release()
```

Panorama stitching manually with SIFT

```
# after SIFT matching img1 -> img2:
src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape((-1,1,2))
dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape((-1,1,2))
Hmat,_ = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)

h1,w1 = img1.shape[:2]
h2,w2 = img2.shape[:2]
canvas = cv2.warpPerspective(img1, Hmat, (w1+w2, max(h1, h2)))
canvas[0:h2, 0:w2] = img2

# easy alternative:
stitcher = cv2.Stitcher_create()
status, pano = stitcher.stitch([img1, img2, img3])
if status == cv2.Stitcher_OK:
```

```
cv2.imwrite("panorama.jpg",pano)
```

Security zone intrusion: polygon test

```
zone = np.array([[100,100],[500,100],[550,400],[80,400]],np.int32)

# for each detected object's center:
cx,cy = 250,220
inside = cv2.pointPolygonTest(zone, (cx,cy), False) >= 0

out = frame.copy()
cv2.polylines(out,[zone],True,(255,0,0),2)
if inside:
    cv2.putText(out,"ALERT",(20,40),
                cv2.FONT_HERSHEY_SIMPLEX,1,(0,0,255),3)
```

HIGH-VALUE APPENDIX - Morphology, Contours, Connected Components

Morphological operations

```
kernel = np.ones((3,3),np.uint8)

eroded = cv2.erode(mask,kernel,iterations=1)
dilated = cv2.dilate(mask,kernel,iterations=1)

opening = cv2.morphologyEx(mask,cv2.MORPH_OPEN,kernel) # remove small white noise
closing = cv2.morphologyEx(mask,cv2.MORPH_CLOSE,kernel) # fill small black holes

gradient = cv2.morphologyEx(mask,cv2.MORPH_GRADIENT,kernel)
tophat = cv2.morphologyEx(gray,cv2.MORPH_TOPHAT,kernel)
blackhat = cv2.morphologyEx(gray,cv2.MORPH_BLACKHAT,kernel)
```

Contours + area/perimeter/bounding box

```
contours,hierarchy = cv2.findContours (
mask,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)

out = img.copy()
for c in contours:
    area = cv2.contourArea(c)
    if area < 100: continue

    peri = cv2.arcLength(c,True)
    x,y,w,h = cv2.boundingRect(c)
    rect = cv2.minAreaRect(c)
    (cx,cy),radius = cv2.minEnclosingCircle(c)

    cv2.drawContours(out,[c],-1,(0,255,0),2)
    cv2.rectangle(out,(x,y),(x+w,y+h),(255,0,0),2)

eps = 0.02*cv2.arcLength(c,True)
approx = cv2.approxPolyDP (c,eps,True)
vertices = len(approx)

M = cv2.moments(c)
if M["m00"] != 0:
    cx = int (M["m10"]/M["m00"])
    cy = int (M["m01"]/M["m00"])

circularity = 4*np.pi*cv2.contourArea(c)/(cv2.arcLength(c,True)**2 + 1e-9)
```

Connected components

```
n, labels, stats, centroids = cv2.connectedComponentsWithStats (mask, 8)
for i in range(1,n): # skip background=0
    x,y,w,h,area = stats[i]
    cx,cy = centroids[i]
    print(i,x,y,w,h,area,cx,cy)
```

Mask arithmetic

```
inter = cv2.bitwise_and(mask1,mask2)
union = cv2.bitwise_or (mask1,mask2)
xor = cv2.bitwise_xor (mask1,mask2)
inv = cv2.bitwise_not (mask1)
```

EXAM-SPEED REFERENCE

Matplotlib grid

```
fig,ax = plt.subplots(1,3,figsize=(15,5))
ax[0].imshow(cv2.cvtColor(img,cv2.COLOR_BGR2RGB)); ax[0].set_title("Original")
ax[1].imshow(gray,cmap="gray"); ax[1].set_title("Gray")
ax[2].imshow(mask,cmap="gray"); ax[2].set_title("Mask")
for a in ax: a.axis("off")
plt.tight_layout(); plt.show()
```

Save image / figure

```
cv2.imwrite("output.png", img) # keep BGR for OpenCV

plt.savefig("figure.png",dpi=300,bbox_inches="tight")

Threshold flags
cv2.THRESH_BINARY
cv2.THRESH_BINARY_INV
cv2.THRESH_TRUNC
cv2.THRESH_TOZERO
cv2.THRESH_TOZERO_INV
cv2.THRESH_OTSU # add with + to binary flag
```

Color conversions

```
cv2.COLOR_BGR2RGB
cv2.COLOR_BGR2GRAY
cv2.COLOR_BGR2HSV
cv2.COLOR_GRAY2BGR
cv2.COLOR_HSV2BGR
```

Common data-type fixes

```
x = img.astype(np.float32) # math without uint8 overflow
x8 = np.clip(x,0,255).astype(np.uint8)

# normalize arbitrary result to 0..255
vis = cv2.normalize(x,None,0,255,cv2.NORM_MINMAX).astype(np.uint8)
```

Image arithmetic

```
add = cv2.add(a,b)
sub = cv2.subtract(a,b)
blend = cv2.addWeighted(a,0.7,b,0.3,0)

# NumPy + on uint8 can wrap; cv2.add saturates at 255
```

Basic image checks

```
print("shape:",img.shape)
print("dtype:",img.dtype)
print("min/max:",img.min(),img.max())
print("mean/std:",img.mean(),img.std())
print("nonzero:",np.count_nonzero(img))
```

Common parameter rules

Gaussian kernel: positive odd sizes, e.g. (3,3),(5,5).

Adaptive blockSize: odd and >1. **C:** subtracted from local mean/weighted mean.

Canny: high threshold usually ~2-3x low as a starting point.

Watershed: markers: background/known regions positive, unknown=0, boundaries after watershed=-1.

K-means: input samples float32 with shape (N, features).

SIFT: requires grayscale commonly; descriptor matching uses NORM_L2; use ratio test.

HoughLinesP: tune threshold, minLineLength, maxLineGap. **HoughCircles:** param1=Canny high threshold, param2=accumulator threshold.

TASK COVERAGE CHECKLIST - What the supplied labs can ask

Lab	Covered material
Lab 01	Python class/dict/functions; image I/O; drawing; ROI/blur; transparent overlay; global/adaptive threshold; rotation; masks/bitwise composition; pixel/channel stats
Lab 02	Histogram; equalization; color maps; gray-world balance; threshold filtering; log/gamma transforms; CT-MRI fusion; image statistics; video capture/write/real-time frame processing
Lab 03	Image coordinates; homogeneous matrices; scaling; translation; rotation; shear; affine mapping; perspective/homography; transform composition
Lab 04	Feature extraction; local/global features; histograms; HOG; LBP; edge-direction/gradient features; texture energy/contrast; filtering; convolution; Sobel; Canny
Lab 05	Global/adaptive/Otsu thresholds; HSV color masks; Canny tuning; region growing; watershed; distance transform; connected components; k-means; compare segmentation methods
Lab 06	Wavelets (CWT/DWT/WPT/IDWT concepts); Sobel/LoG/Canny boundaries; Hough lines/circles; SIFT detect/describe/match; object localization; video recognition; panorama; security-zone logic

Fast decision tree

Dark/bright grayscale issue? histogram/normalize/log/gamma. **Uneven document lighting?** adaptive threshold. **Unknown threshold but bimodal histogram?** Otsu. **Known color?** HSV + inRange. **Need object border?** Canny/Sobel/LoG. **Touching blobs?** morphology + distance transform + watershed. **Need color grouping?** k-means. **Need straight lines?** HoughLinesP. **Need circles?** HoughCircles. **Need robust local matching under scale/rotation?** SIFT. **Need nonperspective alignment/stitching?** SIFT matches + RANSAC.homography + warnPerspective

Exam note: Before submitting: confirm image loaded; print shape/dtype; convert BGR→RGB only for Matplotlib; use grayscale where required; ensure masks are uint8; check writer size; save final output; label plots clearly.